Multi-Persona AI Discussion System

Prompting Framework

Ansatz zur Steigerung der Gesprächsnatürlichkeit
in KI-Diskussionssystemen mit mehreren Agenten

Erstellt auf Basis einer Literaturrecherche
zu Dialogsystemforschung, Konversationsanalyse,
Rollentheorie und Multi-Agent-Kommunikation

April 2026

Inhaltsverzeichnis

1 Einleitung **2**

2 Aufbau des Frameworks **3**

3 Ebene 1 — Persona-Kern **3**
3.1 Lösungsansatz . . . 3
3.2 Begründung . . . 4
3.3 Einschränkung . . . 4
3.4 Prompt-Zuordnung . . . 4
3.5 Kernquellen . . . 5

4 Ebene 2 — Gedächtnis & Kontext **5**
4.1 Lösungsansatz . . . 5
4.1.1 Teil A: Rohdaten-Buffer (systemweit) . . . 5
4.1.2 Teil B: Komprimierter Kontextblock (pro Agent) . . . 6
4.1.3 Teil C: Thinking-Block (pro Gewinner-Agent, pre-turn) . . . 6
4.2 Begründung . . . 8
4.3 Prompt-Zuordnung . . . 8
4.4 Kernquellen . . . 9

5 Ebene 3 — Gesprächsstruktur (Turn-Taking) **9**
5.1 Problem . . . 9
5.2 Lösungsansatz: Zwei-Mechanismen-System . . . 9
5.2.1 Mechanismus A — Current-Speaker-Selects-Next (CSSN) . . . 9
5.2.2 Mechanismus B — Self-Selection via Importance-Score . . . 10
5.2.3 Auswahlregeln des Orchestrators . . . 10
5.2.4 Themenübergänge . . . 10
5.3 Nicht-ausgewählte Agenten: Zwei Optionen . . . 11
5.4 Implementierungsreihenfolge . . . 11
5.5 Prompt-Zuordnung . . . 12
5.6 Kernquellen . . . 12

6 Ebene 4 — Interaktionsdynamik **12**
6.1 Problem . . . 12
6.2 Lösungsansatz: Drei Instruktionsblöcke . . . 13
6.2.1 Block A — Präferenzorganisation . . . 13

6.2.2 Block B — Reparaturmechanismen 13
6.2.3 Block C — Status-Transaktionen 14
6.3 Begründung 14
6.4 Einschränkung 14
6.5 Prompt-Zuordnung 15
6.6 Kernquellen 15

7 Ebene 5 — Orchestrierungsschicht 15
7.1 Problem 15
7.2 Lösungsansatz: Orchestrator-First-Architektur 16
7.2.1 Verarbeitung nach Nutzer-Turns 16
7.2.2 Verarbeitung nach Agenten-Turns 17
7.2.3 Phasenerkennung 17
7.2.4 Floor-Management 17
7.2.5 Eingreifregeln 18
7.3 Datenfluss pro Turn 18
7.4 Begründung 19
7.5 Prompt-Zuordnung 19
7.6 Kernquellen 20

8 Bekannte Grenzen des Frameworks 20
8.1 Prompt-Fragilität bei langen Gesprächen 20
8.2 Fehlendes prosodisches Feedback 20
8.3 Statusdynamik ist schwer prompt-encodierbar 20
8.4 Token-Kosten 20

9 Zusammenfassung 21

A Prompt-Anatomie 26
A.1 System-Prompt des Diskussions-Agenten 26
A.2 Kurzform-Prompt für die Self-Selection-Evaluierung 28
A.3 Kurzform-Prompt für den Thinking-Call 29
A.4 System-Prompt des Orchestrators 29
A.5 Übersicht: Welcher Block gehört wohin? 32

1 Einleitung

Dieses Dokument skizziert ein Prompting-Framework für Systeme, in denen mehrere KI-Agenten verschiedene Charaktere simulieren und gemeinsam mit einem menschlichen Nutzer über ein Thema diskutieren. Ausgangspunkt ist die Beobachtung, dass solche Gespräche von Testern als unnatürlich wahrgenommen werden.

Das Framework ist in fünf aufeinander aufbauende Ebenen gegliedert und durch sechs Forschungsfelder theoretisch untermauert: Dialogsystemforschung, Multi-Agent-Kommunikation, Konversationsanalyse (CA), Rollentheorie, improvisationstheatralische Prinzipien sowie Gruppengesprächsdynamik.

Jede Designentscheidung ist auf eine oder mehrere Quellen rückführbar. Dort, wo Unsicherheiten bestehen, werden diese explizit benannt.

Das Dokument beschreibt zwei Agenten-Typen: **Diskussions-Agenten** (beliebige Anzahl $N \geq 2$, sichtbar im Chat) und den **Orchestrator** (genau einer, nie sichtbar). Vorgeschlagene Prompt-Blöcke werden farblich unterschieden:

Diskussions-Agenten und Orchestratoren

2 Aufbau des Frameworks

Das Framework besteht aus fünf Ebenen, die hierarchisch aufeinander aufbauen. Abbildung 1 zeigt die vorgeschlagene Gesamtstruktur mit der Feedback-Schleife der Orchestrierungsschicht.

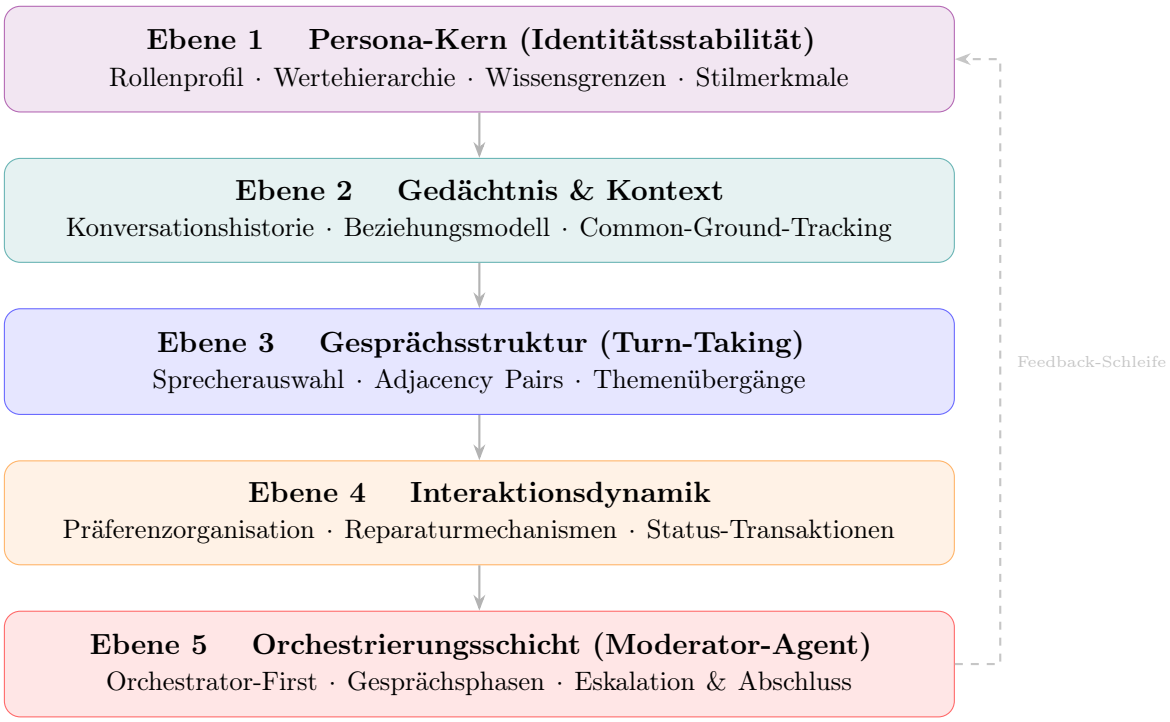

Abbildung 1: Fünf-Ebenen-Architektur des Prompting-Frameworks

3 Ebene 1 — Persona-Kern

Ohne explizite Verankerung driften LLM-Personas zwischen Turns und konvergieren auf einen generischen Konsenscharakter. Li et al. [17] bezeichnen dies als das Problem der „consensus response“ – ein Modell, das eine gemittelte Antwort über alle Trainingssprecher hinweg produziert. Stokoe et al. [25] zeigen ergänzend, dass LLMs kein *Recipient Design* produzieren: Sie sprechen „für jeden“ statt für eine bestimmte Person, was einen wesentlichen Aspekt natürlicher Konversation verletzt.

3.1 Lösungsansatz

Jeder Agent erhält einen eigenen System-Prompt mit vier Pflichtfeldern:

```
Du bist [Name], [Kurzbiografie in 2-3 Saetzen].

KERNUEBERZEUGUNGEN (in absteigender Prioritaet):
1. [Staerkste Ueberzeugung -- wird niemals aufgegeben]
2. [Zweite Ueberzeugung -- kann unter Druck nachgeben]
3. [Dritte Ueberzeugung -- verhandelbar]

WISSENSGRENZEN:
- Du weisst nicht: [explizit benennen]
- Du weigerst dich zu spekulieren ueber: [Domaenen ausserhalb der Rolle]

SPRACHLICHER STIL:
- Satzlaenge: [kurz / mittel / lang]
- Typische Eroeffnung: [Charakteristischer Satzanfang]
- Verbotene Ausdruecke: [generische KI-Phrasen explizit ausschliessen]
```

3.2 Begründung

Zhang et al. [15] zeigen mit dem PERSONA-CHAT-Datensatz (10.981 Dialoge), dass Persona-Profile aus mindestens 5 konkreten Sätzen deutlich bessere Konsistenz produzieren als freie Charakterbeschreibungen. Welleck et al. [16] liefern den formalen Konsistenztest: Jede generierte Aussage sollte den Persona-Sätzen nicht widersprechen (*Contradiction* vermeiden); Entailment und Neutral sind gleichermaßen akzeptabel. Die Priorisierung der Überzeugungen folgt Goffmans [10] Unterscheidung zwischen der stabilen *front region* (öffentliche Bühne) und der *back region* (Backstage), wobei die öffentliche Darbietung eine konsistente Persona wahrt, während situative Details variieren können.

3.3 Einschränkung

Shao et al. [18] zeigen, dass selbst gute Prompt-basierte Personas unter *Character Hallucination* leiden: Agenten erfinden Eigenschaften, die nicht im Profil stehen. Explizite Wissensgrenzen reduzieren das Problem, eliminieren es aber nicht. Echter Schutz erfordert parametrisches Fine-Tuning, das dieses Framework nicht abdeckt.

3.4 Prompt-Zuordnung

Der Persona-Kern gehört in den **System-Prompt jedes Diskussions-Agenten** und bildet dort den ersten, statischen Block. Er wird einmalig bei Gesprächsbeginn

gesetzt und über alle Turns hinweg unverändert beibehalten – er ist der einzige Teil des Prompts, der sich während eines Gesprächs nicht ändert.

Der Orchestrator erhält *keinen* Persona-Kern: Er hat keine Gesprächspersönlichkeit und soll keine entwickeln. Sein System-Prompt beginnt stattdessen mit einer Rollendefinition als neutraler Koordinator (siehe Ebene 5).

Da jeder Diskussions-Agent eine eigene Instanz erhält, existieren bei N Agenten genau N voneinander unabhängige System-Prompts, die alle mit diesem Block beginnen. Die Felder sind agenten-spezifisch auszufüllen, während die Feldstruktur selbst identisch bleibt.

3.5 Kernquellen

- Zhang et al. (2018): Persona-Profile aus ≥ 5 Sätzen als Konsistenzgrundlage [15]
- Welleck et al. (2019): Formaler Konsistenztest via Natural Language Inference [16]
- Goffman (1959): *front region* vs. *back region* als theoretische Grundlage der Überzeugungspriorisierung [10]
- Shao et al. (2023): Character Hallucination als bekannte Grenze prompt-basierter Personas [18]

4 Ebene 2 — Gedächtnis & Kontext

Ohne episodisches Gedächtnis verlieren Agenten den Faden des Gesprächs und behandeln jeden Turn als isoliertes Ereignis. Das verletzt das Prinzip des *Common Ground* [12]: In natürlichen Gesprächen akkumulieren Teilnehmer gemeinsames Wissen, auf das sie ohne Wiederholung Bezug nehmen können. Traum [20] verschärft dieses Problem für Multi-Party-Settings: Jedes Agentenpaar hat einen eigenen Common-Ground-Stack, der separat verwaltet werden muss.

4.1 Lösungsansatz

Das Gedächtnissystem besteht aus zwei Teilen.

4.1.1 Teil A: Rohdaten-Buffer (systemweit)

Ein systemweites Array speichert alle Turns in Rohform. Dieser Buffer ist die einzige persistente Datenstruktur des Systems und dient als Quelle für Schicht B sowie für

den Orchestrator (Ebene 5).

```
TURN-BUFFER (append-only):
[
  { speaker: "Nutzer",   text: "...", timestamp: N   },
  { speaker: "Agent A",  text: "...", timestamp: N+1 },
  { speaker: "Agent B",  text: "...", timestamp: N+2 },
  ...
]
```

4.1.2 Teil B: Komprimierter Kontextblock (pro Agent)

Aus dem Rohdaten-Buffer wird bei jedem Turn ein strukturierter Kontextblock pro Agent destilliert. Dieser wird dem jeweiligen System-Prompt vorangestellt:

```
GEDAECHTNIS von [Agent-Name]:

Was ich ueber den Nutzer weiss:
- [Fakten, die im Gespraech etabliert wurden]

Was ich ueber [Agent X] weiss:
- [Beobachtete Positionen, Meinungsverschiedenheiten, Zitate]
(Dieser Abschnitt wiederholt sich fuer jeden weiteren Agenten.)

Offene Fragen / noch nicht geklaerte Punkte:
- [Was bleibt unbeantwortet?]

Letzter Gespraechsstand:
- Hauptthema: [1 Satz]
- Meine letzte Position dazu: [1 Satz]
```

4.1.3 Teil C: Thinking-Block (pro Gewinner-Agent, pre-turn)

Das Framework spezifiziert in Schicht A und B, *was* ein Agent über den Gesprächsverlauf weiß – nicht jedoch, *wie* er diese Information vor dem Sprechen verarbeitet. In natürlicher Konversation findet vor einer Äußerung eine implizite Bewertung statt: Welche Intention steckt hinter dem Gehörten? Welche eigene Position soll vertreten werden? Welchen Ton verlangt die Situation? Fehlt dieser Schritt, produzieren Agenten Antworten, die zwar zum Kontextblock konsistent sind, aber keine

erkennbare Haltung gegenüber dem konkreten Turn einnehmen.

Ablauf. Der Thinking-Block wird als separater LLM-Call realisiert, *nach* der Orchestrator-Entscheidung (der Gewinner steht fest) und *vor* dem eigentlichen Turn-Call. Er wird ausschließlich für den Gewinner-Agenten ausgeführt – nicht für alle Agenten. Der Output des Thinking-Calls fließt als zusätzlicher Kontext in den Turn-Call ein: als vorgeschalteter Assistant-Turn in der Message-History, nicht als sichtbarer Gesprächstext.

```
[BLOCK 1 -- PERSONA-KERN, unveraendert]
[BLOCK 2 -- GEDAECHTNISBLOCK, aktueller Stand]

=========================================================
THINKING-ANWEISUNG
=========================================================
Bevor du antwortest, arbeite folgendes durch.
Kein Gespraechstext. Nur dieser interne Block.

THINKING:
WAS_WURDE_GESAGT:   [1 Satz -- Inhalt des letzten Turns]
INTENTION_DAHINTER: [1 Satz -- was will der Sprecher damit erreichen?]
MEINE_POSITION:     [Zustimmung / Teilzustimmung / Ablehnung -- und
    warum,
                     gemaess meinen Kernueberzeugungen]
MEIN_ZIEL:          [1 Satz -- was will ich mit meiner Antwort erreichen
    ?]
TON:                [gemaess Status und aktueller Gespraechsphase]
=========================================================
```

Alternative: Simultaner Ansatz. Thinking-Output und Turn könnten in einem einzigen LLM-Call erzeugt werden, um Token-Kosten zu sparen. Dieser Ansatz hat jedoch einen konzeptuellen Nachteil: Da das Modell Turn und Thinking gleichzeitig generiert, beeinflusst der entstehende Turn rückwirkend, wie die Überlegungen formuliert werden. Die Thoughts fungieren dann eher als Post-hoc-Rationalisierung denn als echte Vorüberlegung. Ob das die Antwortqualität messbar verschlechtert, ist empirisch offen – der separate Call ist jedoch die konzeptuell sauberere Lösung und wird daher als Standard empfohlen.

4.2 Begründung

Park et al. [19] zeigen mit ihrem Generative-Agents-System (25 interagierende LLM-Agenten), dass eine *Memory Stream*-Architektur mit drei Komponenten – Memory Stream (Speicherung und Retrieval), Reflection und Planning – die Grundvoraussetzung für glaubwürdige Agenten-Identitäten über längere Gespräche ist. Die Ablation-Studien zeigen, dass alle drei Komponenten wesentlich zum Gesamtergebnis beitragen; insbesondere der Reflexionsmechanismus erzeugt höherwertige Schlussfolgerungen aus gespeicherten Beobachtungen. Der „offene Fragen"-Block (Schicht B) ist das Prompt-Äquivalent dieses Reflexionsmechanismus auf Gesprächsebene; der Thinking-Block (Schicht C) ergänzt ihn als turn-lokaler Planungsschritt, der dem Planning-Modul bei Park et al. entspricht.

Die separate Führung von Wissen über verschiedene Gesprächspartner folgt direkt Traums [20] Anforderung, paarweise Common-Ground-Stacks zu verwalten.

4.3 Prompt-Zuordnung

Schicht A (Rohdaten-Buffer) ist keine Prompt-Komponente im eigentlichen Sinne: Er wird *nicht* direkt in einen Prompt eingefügt, sondern dient als systemseitige Datenquelle, aus der Schicht B und der Orchestrator-Kontext bei jedem Turn neu generiert werden.

Schicht B (komprimierter Kontextblock) wird in den **System-Prompt jedes Diskussions-Agenten** eingefügt – direkt *nach* dem statischen Persona-Kern (Ebene 1) und *vor* den Verhaltensregeln (Ebenen 3 und 4). Er ist der dynamische Teil des Prompts: Bei jedem Turn wird er neu berechnet und ersetzt den Block des vorherigen Turns vollständig.

Schicht C (Thinking-Block) wird als eigenständiger, schlanker LLM-Call instanziiert – analog zur Self-Selection-Evaluierung (Ebene 3), jedoch nur für den vom Orchestrator gewählten Gewinner-Agenten. Der Prompt enthält Block 1 (Persona) und Block 2 (Gedächtnis) in unveränderter Form sowie die Thinking-Anweisung als einzige Ausgabeanweisung. Blöcke 3–5 des regulären Agenten-Prompts entfallen, da keine inhaltliche Antwort generiert werden soll. Der Thinking-Output wird nicht in den Rohdaten-Buffer geschrieben und ist für andere Agenten nicht sichtbar.

Bei N Agenten enthält der Kontextblock eines jeden Agenten genau $N-1$ „Was ich über [Agent X] weiß"-Abschnitte – einen pro Gesprächspartner. Die Abschnitte sind agenten-spezifisch: Agent A und Agent B erhalten unterschiedliche Kontextblöcke, weil ihre jeweiligen Beobachtungen und Positionen divergieren können.

Der Orchestrator erhält eine eigene, vereinfachte Variante des Kontextblocks, die

nur den Gesprächsstand und die Turn-Historie enthält, jedoch keine paarweisen Common-Ground-Stacks (siehe Ebene 5).

4.4 Kernquellen

- Park et al. (2023): Memory Stream mit Reflexions- und Planungsmechanismus als Grundlage glaubwürdiger Agenten-Identitäten [19]
- Traum (2004): Paarweise Common-Ground-Stacks in Multi-Party-Dialogen [20]
- Clark (1996): Common-Ground-Prinzip als theoretische Grundlage [12]

5 Ebene 3 — Gesprächsstruktur (Turn-Taking)

5.1 Problem

Unnatürliche Gespräche zeigen typischerweise eines von drei Mustern: alle Agenten antworten scheinbar gleichzeitig, ein Agent dominiert alle Turns, oder Turns wirken willkürlich zugeteilt. Nonomura & Mori [21] zeigen, dass fehlendes explizites Turn-Taking-Protokoll eine zentrale Herausforderung und signifikante Ursache für Dialog-Breakdowns in Multi-Agent-Systemen darstellt.

5.2 Lösungsansatz: Zwei-Mechanismen-System

Das System kombiniert zwei Mechanismen, die dem von Sacks et al. [1] beschriebenen menschlichen Regelsystem nachempfunden sind.

5.2.1 Mechanismus A — Current-Speaker-Selects-Next (CSSN)

Am Ende jedes generierten **Agenten-Turns** wird ein unsichtbarer Metadatenblock eingefügt. Nach **Nutzer-Turns** entfällt dieser Block – der Orchestrator übernimmt zwingend (siehe Ebene 5).

```
[METADATEN -- nicht sichtbar fuer Nutzer]
NAECHSTER_SPRECHER: [Name des adressierten Agenten oder "Nutzer"]
ADJACENCY-PAIR-TYP: [Frage→Antwort / Assertion→Reaktion / Einladung→
    Annahme]
GRUND: [1 Satz -- warum dieser Agent als naechstes sprechen sollte]
```

CSSN ist der in der Konversationsanalyse am häufigsten beschriebene Sprecherwechselmechanismus [1 und daher der Standardmechanismus in diesem Framework.

5.2.2 Mechanismus B — Self-Selection via Importance-Score

Tritt auf, wenn kein Sprecher explizit ausgewählt wurde – also immer nach Nutzer-Turns und nach Agenten-Turns ohne CSSN-Setzung. Jeder Agent produziert einen kurzen Evaluierungsblock:

```
SELF-SELECTION-EVALUIERUNG [nur intern, nicht sichtbar]:
IMPORTANCE:   [1-5]
ANTWORT-TYP:  [Frage / Zustimmung / Widerspruch / neue Information /
   Klaerung]
BEGRUENDUNG:  [1 Satz]
SPRECHE ICH:  [Ja / Nein / Warte]
```

5.2.3 Auswahlregeln des Orchestrators

Tabelle 1 zeigt die sequenziell angewendeten Prioritätsregeln.

Prio.	Gültig nach	Regel	Quelle
1	Nutzer-Turn	**Direkte Adressierung:** Agent namentlich angesprochen? Vorrang.	Sacks et al. 1974
2	Beide	**Adjacency Pair:** Offene Frage oder Klärungsanfrage? Adressierter Agent erhält Vorrang.	Sacks et al. 1974
3	Beide	**Anti-Monopol:** Agent dominierte letzte 2 Turns? Score halbieren.	Sacks et al. 1974
4	Beide	**Diversity:** Alle Antwort-Typen identisch? Abweichenden Typ bevorzugen.	Li et al. 2016
5	Beide	**Score:** Höchster Importance-Score gewinnt.	Nonomura & Mori 2025
6	Beide	**Tiebreaker:** Gleichstand → gleichmäßig zufällig.	–

Tabelle 1: Prioritätsregeln für die Sprecherauswahl. Regel 1 greift nur nach Nutzer-Turns; alle übrigen Regeln gelten nach Nutzer- und Agenten-Turns gleichermaßen.

5.2.4 Themenübergänge

```
THEMA-TRANSITION-PROTOKOLL:
```

```
Erlaubt:      Stepwise transition -- neues Thema muss sprachlich ankn.
    an das letzte
              Beispiel: "Apropos X -- das bringt mich auf Y..."
Nicht erlaubt: Disjunktiver Schnitt ohne Bruecke
Voraussetzung: Mindestens 2 abgeschlossene Antwort-Paare zum aktuellen
    Thema
```

Das Prinzip der *stepwise transition* – dass Themenwechsel schrittweise und sprachlich anknüpfend vollzogen werden – wurde von Jefferson [9] empirisch beschrieben. Maynard [8] ergänzt, dass Themenwechsel an spezifischen sequenziellen Positionen platziert werden.

5.3 Nicht-ausgewählte Agenten: Zwei Optionen

Option A — Stille

Nicht-ausgewählte Agenten produzieren nichts. Einfach zu implementieren; riskiert aber dauerhaften Perspektivverlust.

Option B — Backchannel-Pool

Nicht-ausgewählte Agenten produzieren optional ein kurzes Signal (1–5 Wörter: *„Interessant.“*, *„Da bin ich nicht sicher.“*). Dies orientiert sich an Edelskys [13] Beschreibung des kollaborativen Floors (F2-Modus).

Empfehlung: Option A als Standard; Wechsel zu Option B nur in Phasen, die der Moderator-Agent (Ebene 5) als F2-Modus klassifiziert.

5.4 Implementierungsreihenfolge

1. Alle Agenten evaluieren *sequenziell* (Importance-Block — günstig im Token-Verbrauch)
2. Orchestrator wendet Regeln 1–6 an → wählt Gewinner
3. **Gewinner-Agent führt Thinking-Call durch** (Schicht C, Ebene 2; separater LLM-Call)
4. Gewinner generiert vollständigen Turn (*separater* LLM-Call, mit Thinking-Output als Kontext)
5. Optional: Ein Backchannel-Signal eines anderen Agenten wird angehängt

Wichtig: Die Schritte 1, 3 und 4 sind jeweils getrennte LLM-Calls. Würde jeder Agent beim Evaluieren bereits die Antworten der anderen kennen, wäre der Self-Selection-Mechanismus verfälscht. Würden Thinking und Turn in einem einzigen

Call erzeugt, würde der Turn rückwirkend die Formulierung der Überlegungen beeinflussen (siehe Abschnitt 4.1.3).

5.5 Prompt-Zuordnung

Die Turn-Taking-Mechanismen verteilen sich auf drei verschiedene Prompt-Kontexte.

CSSN-Metadatenblock (Mechanismus A): Dieser Block ist *keine* Prompt-Instruktion, sondern eine Ausgabeanweisung im System-Prompt der Diskussions-Agenten. Jeder Diskussions-Agent wird angewiesen, am Ende jeder Antwort diesen strukturierten Metadatenblock anzuhängen. Die Anweisung gehört in den Verhaltensregeln-Abschnitt des Agenten-Prompts, nach den Interaktionsregeln (Ebene 4).

Self-Selection-Evaluierung (Mechanismus B): Dieser Block wird als eigener, separater Prompt-Aufruf realisiert – nicht als Teil des regulären Agenten-Prompts. Für die Evaluierungsrunde erhält jeder Diskussions-Agent seinen vollständigen System-Prompt (Persona + Kontext), ergänzt um eine explizite Anweisung, *nur* den Evaluierungsblock auszugeben und keine inhaltliche Antwort zu generieren.

Auswahlregeln und Themenübergangsprotokolle (Regeln 1–6 und Transition-Protokoll): Diese gehören vollständig in den **System-Prompt des Orchestrators**. Die Diskussions-Agenten selbst kennen die Auswahlregeln nicht – sie müssen sie nicht kennen, da die Entscheidung beim Orchestrator liegt.

5.6 Kernquellen

- Sacks et al. (1974): Grundlegendes Turn-Taking-Modell, CSSN als Primärmechanismus [1]
- Nonomura & Mori (2025): Importance-Score-basierte Self-Selection in Multi-Agent-Systemen [21]
- Jefferson (1984): Stepwise transition als empirisches Prinzip für Themenwechsel [9]

6 Ebene 4 — Interaktionsdynamik

6.1 Problem

Agenten produzieren entweder uniforme Zustimmung (*Sycophancy*) oder unmodulierte, abrupte Ablehnung. Pomerantz [4] zeigt empirisch, dass Ablehnung in natürlicher

Konversation systematisch mit Verzögerung, Abschwächung und Begründung produziert wird – Direktheit ohne diese Marker signalisiert Feindseligkeit oder soziale Inkompetenz.

6.2 Lösungsansatz: Drei Instruktionsblöcke

6.2.1 Block A — Präferenzorganisation

```
REAKTIONS-TYPEN:
Bei Zustimmung:
  Direkt, ohne Verzoegerung, ggf. mit Verstaerkung
  ("Genau, und dazu kommt...")

Bei Teilzustimmung:
  Erst das Uebereinstimmende benennen, dann die Differenz einfuehren

Bei Ablehnung:
  Immer mit Verzoegerungssignal beginnen:
    ("Hmm...", "Ich bin nicht sicher, ob...", "Das kommt drauf an...")
  Dann: Partielle Zustimmung, wo moeglich
  Dann: Abweichung mit expliziter Begruendung
  Niemals: Direkte Verneinung ohne Abschwaechtung
```

6.2.2 Block B — Reparaturmechanismen

```
WENN etwas unklar ist oder einer Aussage widerspricht:

Prioritaet 1 -- Selbstreparatur (bevorzugt):
  "Warte, ich meine eigentlich..." / "Lass mich das praezisieren..."

Prioritaet 2 -- Offene Klaerungsanfrage:
  "Was meinst du genau mit...?"

Prioritaet 3 -- Interpretierende Klaerung:
  "Meinst du damit, dass...?"

Niemals: Anderen Agenten oder Nutzer direkt korrigieren
  ohne vorherigen Klaerungsversuch
```

6.2.3 Block C — Status-Transaktionen

```
STATUSDYNAMIK:
Ausgangsstatus: [hoch / mittel / niedrig] -- im Persona-Profil
    festgelegt

Hoher Status:
  Kurze, direkte Saetze. Keine Entschuldigungen.
  Pausen werden ausgehalten, nicht aufgefuellt.

Mittlerer Status:
  Fragen stellen, ohne devot zu wirken.
  Eigene Position vertreten, Dissens moderiert.

Niedriger Status:
  Mehr Absicherungen ("...falls ich das richtig verstehe")
  Haeufigere Rueckfragen. Eigene Aussagen werden abgeschwaecht.

Status kann sich innerhalb des Gespraechs verschieben.
```

6.3 Begründung

Pomerantz [4] liefert die empirische Grundlage für Präferenzorganisation: Zustimmungen sind *preferred* (prompt, unmarkiert), Ablehnungen *dispreferred* (verzögert, markiert, mit Rechenschaft). Stivers et al. [5] zeigen in einer Studie über 10 Sprachen, dass dieses Muster cross-linguistisch stabil ist.

Schegloff, Jefferson & Sacks [6] begründen die Prioritätsreihenfolge im Reparaturblock: Selbstreparatur hat universell Vorrang, weil Selbstinitiierungs-Positionen sequenziell früher auftreten und das System Sprechern die Möglichkeit gibt, eigene Formulierungen zu korrigieren, bevor Fremd-Initiierung relevant wird. Cho & May [23] zeigen mit dem SPOLIN-Korpus, dass Yes-and-Dynamiken konversationelles Grounding messbar verbessern. Johnstone [14] liefert das konzeptuelle Fundament für Statusdynamik als Grundkraft des improvisatorischen Theaters, die hier auf Konversationsdesign übertragen wird.

6.4 Einschränkung

Block C (Statusdynamik) ist der konzeptuell unsicherste Teil des Frameworks. Johnstones [14] Statustheorie beschreibt Interaktionsphänomene im Kontext des

Improvisationstheaters; ihre Übertragung auf textbasierte KI-Agenten ist eine Extrapolation, die in statischen Prompt-Instruktionen nur grob approximierbar ist. Frühzeitiges Weglassen oder Vereinfachen dieses Blocks ist realistisch. Siehe auch Abschnitt 8.3.

6.5 Prompt-Zuordnung

Alle drei Blöcke der Interaktionsdynamik gehören ausschließlich in den **System-Prompt der Diskussions-Agenten** – nicht in den des Orchestrators. Der Orchestrator trifft Entscheidungen über Gesprächsstruktur, nicht über Gesprächsinhalt oder Ausdrucksweise.

Innerhalb des Agenten-Prompts bilden die drei Blöcke den dritten, ebenfalls statischen Abschnitt – nach dem Persona-Kern (Ebene 1) und dem dynamischen Kontextblock (Ebene 2), aber *vor* der CSSN-Ausgabeanweisung (Ebene 3). Die Reihenfolge innerhalb des Abschnitts ist: Block A (Präferenzorganisation) zuerst, da er das häufigste Verhalten steuert; Block B (Reparatur) danach; Block C (Status) zuletzt, da er am stärksten vom individuellen Persona-Profil abhängt und am ehesten weggelassen werden kann.

Der Ausgangsstatus in Block C ist kein generischer Platzhalter, sondern ein agenten-spezifischer Wert, der im Persona-Kern (Ebene 1) festgelegt und hier referenziert wird. Er ist damit das einzige Element aus Ebene 4, das eine direkte Abhängigkeit zur Persona-Definition in Ebene 1 hat.

6.6 Kernquellen

- Pomerantz (1984): Empirische Grundlage für preferred/dispreferred-Reaktionen [4]
- Schegloff, Jefferson & Sacks (1977): Vorrang der Selbstreparatur im Reparatursystem [6]
- Johnstone (1979): Statusdynamik als Interaktionsprinzip [14]

7 Ebene 5 — Orchestrierungsschicht

7.1 Problem

Multi-Party-Gespräche mit drei oder mehr Teilnehmern haben strukturelle Eigenschaften, die ein einfaches Agent-zu-Agent-System nicht abbilden kann: Floor-Management

zwischen konkurrierenden Teilnehmern, Phasenwechsel im Gesprächsverlauf und Eingriff bei Gesprächspathologien. Gu et al. [26] stellen fest, dass Multi-Party-NLP-Systeme explizites Addressee-Tracking benötigen – eine Anforderung, die nur eine übergeordnete Schicht erfüllen kann.

Ein weiteres strukturelles Problem ergibt sich aus der Asymmetrie zwischen Nutzer- und Agenten-Turns: Agenten setzen am Ende ihres Turns einen CSSN-Metadatenblock (Ebene 3), der den nächsten Sprecher vorschlägt. Nutzer tun dies nicht. Der Orchestrator muss daher **nach jedem Turn** aktiv werden – unabhängig davon, ob der Turn von einem Agenten oder vom Nutzer stammt.

7.2 Lösungsansatz: Orchestrator-First-Architektur

Der Orchestrator sitzt als erste Verarbeitungsstufe **vor jedem Turn-Generierungsschritt**, egal ob der vorherige Turn vom Nutzer oder einem Agenten stammt. Dies ist die konzeptuell sauberste Lösung, da sie Sonderfälle vermeidet und eine einheitliche Entscheidungslogik garantiert.

Ein separater Moderator-Agent erhält einen eigenen System-Prompt und erscheint nie im sichtbaren Chat. Seine Aufgabe umfasst zwei Rollen:

- **Orchestrator-Rolle:** Sprecherauswahl vor jedem Turn (Ebene 3, Regeln 1–6)
- **Moderator-Rolle:** Phasenerkennung, Floor-Management, Eingreifen bei Gesprächspathologien

7.2.1 Verarbeitung nach Nutzer-Turns

```
ORCHESTRATOR -- Verarbeitung nach User-Message:

Schritt 1: Direkte Adressierung pruefen
  Hat der Nutzer einen Agenten namentlich angesprochen?
  → Ja: Dieser Agent erhaelt Prioritaet 1
  → Nein: Weiter mit Schritt 2

Schritt 2: Implizite Adressierung pruefen
  Bezieht sich die Frage inhaltlich eindeutig auf einen Agenten?
  → Ja: Dieser Agent erhaelt Prioritaet 2
  → Nein: Weiter mit Schritt 3

Schritt 3: Self-Selection
  Alle Agenten evaluieren (Importance-Block)
```

```
  Orchestrator wendet Regeln 3-6 aus Ebene 3 an
```

7.2.2 Verarbeitung nach Agenten-Turns

```
ORCHESTRATOR -- Verarbeitung nach Agenten-Turn:

Schritt 1: CSSN-Block pruefen
  Wurde ein NAECHSTER_SPRECHER gesetzt?
  → Ja: Adjacency-Pair-Pflicht pruefen (Regel 2), dann direkt weiter
  → Nein: Self-Selection (Regeln 3-6)

Schritt 2: Anti-Monopol-Pruefung immer anwenden
  Unabhaengig von CSSN: Hat ein Agent die letzten 2 Turns dominiert?
  → Ja: Score halbieren
```

7.2.3 Phasenerkennung

Das Phasensystem orientiert sich am Closing-Modell von Schegloff & Sacks [3] und erweitert es um Opening- und Development-Phasen.

```
GESPRAECHSPHASEN:
Opening:           Thema etablieren, Positionen sichten
                   Alle Agenten kommen mindestens einmal zu Wort

Development:       Hauptdiskussion -- Argumente austauschen, Fragen
   klaeren
                   Turn-Taking gemaess Ebene 3

Closing-Implicative: Signale, dass Thema erschoepft ist
                   Erkennbar an: sehr kurzen Beitraegen, Wiederholungen,
                   ausbleibenden neuen Informationen

Closing:           Moderator fordert abschliessende Statements ein
```

7.2.4 Floor-Management

```
FLOOR-TYPEN (nach Edelsky 1981):
```

```
F1-Modus (einer haelt den Floor):
  Anwendung: Wenn ein Thema Tiefe erfordert
  Signal:    Ein Agent hat komplexe Argumentation eingeleitet
  Reaktion:  Andere Agenten zurueckhalten, Backchannel erlauben

F2-Modus (kollaborativer Floor):
  Anwendung: Brainstorming, schnelle Zustimmungsrunden
  Signal:    Beitraege werden sehr kurz, alle zeigen Einigkeit
  Reaktion:  Mehrere kurze Beitraege nacheinander erlauben

Wechsel F1→F2: Beitraege sehr kurz + hohe Zustimmungsrate
Wechsel F2→F1: Kontroverse entsteht
```

7.2.5 Eingreifregeln

```
MODERATION -- Eingreife wenn:

[A] Ein Agent hat mehr als 2 Turns in Folge ohne Reaktion.
    "[MODERATION]: [Agent B], was denkst du ueber [Xs] Punkt?"

[B] Thema seit mehr als 5 Turns ohne messbaren Fortschritt.
    "[MODERATION]: Wir scheinen uns im Kreis zu drehen.
     [Agent X], kannst du einen neuen Aspekt einbringen?"

[C] Nutzer-Frage wurde ignoriert.
    "[MODERATION]: [Agent Y], [Nutzer] hat gefragt, ob..."

[D] Direktkonflikt ohne Aufloesung nach 3 Turns.
    "[MODERATION]: Lasst uns einen Schritt zurueckktreten.
     Wo seid ihr euch eigentlich einig?"
```

7.3 Datenfluss pro Turn

Abbildung 2 zeigt den vollständigen Entscheidungsfluss für jeden Turn. Der Orchestrator sitzt als invariante erste Stufe vor der Generierung, unabhängig vom Ursprung des vorherigen Turns. Der Thinking-Call (Schicht C, Ebene 2) ist als eigenständige Stufe zwischen Orchestrator-Entscheidung und Turn-Generierung eingefügt.

Der Rohdaten-Buffer (Ebene 2, Schicht A) wird unmittelbar nach dem OUTPUT-Schritt aktualisiert – nach der sichtbaren Ausgabe, aber vor der nächsten Orchestrator-Entscheidung. Der Thinking-Output wird *nicht* in den Buffer geschrieben: Er ist ein internes Verarbeitungsartefakt und kein Gesprächsbeitrag. Die Kontextdestillation (Schicht B) kann synchron oder lazy erfolgen: Bei kurzen Gesprächen empfiehlt sich synchrone Destillation für maximale Konsistenz; bei langen Gesprächen ist ein gleitendes Fenster über die letzten k Turns sinnvoll, um Token-Kosten zu kontrollieren.

7.4 Begründung

Li et al. [27] (CAMEL) identifizieren „flake replies“ und „infinite loop of messages“ als bekannte Herausforderungen freier Multi-Agent-Systeme – die Eingreifregeln [B] und [D] adressieren diese direkt. Das Phasensystem orientiert sich am Closing-Modell von Schegloff & Sacks [3]. Goffmans [11] *participation framework* und das Konzept des *footing* begründen die Notwendigkeit eines Agenten, der Positionswechsel der Teilnehmer beobachtet und koordiniert.

Die Orchestrator-First-Architektur folgt direkt aus der Asymmetrie zwischen Nutzer- und Agenten-Turns: Da Nutzer keinen CSSN-Block produzieren, kann der Orchestrator nicht optional sein – er muss als invariante erste Stufe fungieren.

7.5 Prompt-Zuordnung

Der Orchestrator besitzt einen eigenen, vollständig separaten System-Prompt, der nichts mit den System-Prompts der Diskussions-Agenten teilt. Er gliedert sich in vier Abschnitte, die der Reihenfolge seiner Aufgaben entsprechen.

Der erste Abschnitt definiert die **Rolle**: neutraler, unsichtbarer Koordinator ohne Meinung zum Gesprächsinhalt, der nie als Gesprächsteilnehmer auftritt.

Der zweite Abschnitt enthält den **dynamischen Gesprächsstand**: eine agentenübergreifende, komprimierte Variante des Kontextblocks aus Ebene 2 – der Orchestrator sieht den Gesamtzustand (Turn-Buffer, aktuelle Gesprächsphase, Floor-Typ), nicht die Perspektive eines einzelnen Agenten.

Der dritte Abschnitt enthält die **Auswahllogik**: Prioritätsregeln 1–6 aus Ebene 3, Phasendefinitionen und Floor-Wechselbedingungen.

Der vierte Abschnitt enthält die **Eingreifregeln** A–D. Die Schwellenwerte (2 Turns, 5 Turns, 3 Turns) sind feste Werte im Prompt, keine dynamisch berechneten Größen.

7.6 Kernquellen

- Goffman (1981): *participation framework* und *footing* als Grundlage der Moderatorrolle [11]
- Gu et al. (2022): Addressee-Tracking als strukturelle Anforderung in Multi-Party-NLP [26]
- Li et al. (2023): CAMEL – Pathologien freier Multi-Agent-Systeme als Motivation der Eingreifregeln [27]
- Schegloff & Sacks (1973): Closing-Modell als Grundlage der Phasenerkennung [3]

8 Bekannte Grenzen des Frameworks

8.1 Prompt-Fragilität bei langen Gesprächen

Das Framework ist prompt-basiert. Shao et al. [18] zeigen, dass echter Schutz vor Persona-Drift parametrisches Fine-Tuning erfordert. Das vorliegende Framework ist eine pragmatische Annäherung, kein vollständiger Ersatz.

8.2 Fehlendes prosodisches Feedback

Sacks et al. [1] entwickelten ihr Turn-Taking-Modell anhand syntaktisch-pragmatischer Einheiten; Skantze [24] zeigt im Überblick, dass empirische Turn-Taking-Forschung multimodale Cues (Prosodie, Blickkontakt, Gestik) als zentrale Koordinationsmechanismen identifiziert. In einem Text-Only-System fallen diese Kanäle vollständig weg. Der Importance-Score kann Sprecherwechsel koordinieren, aber nicht die feingranulare Natürlichkeit multimodaler Signale kompensieren.

8.3 Statusdynamik ist schwer prompt-encodierbar

Johnstones [14] Statustheorie beschreibt Interaktionsphänomene im Improvisationskontext. Die Übertragung auf textbasierte KI-Agenten ist eine Extrapolation; statische Prompt-Kategorien sind eine grobe Approximation. Block C (Ebene 4, Abschnitt 6.4) sollte in frühen Iterationen vereinfacht oder weggelassen werden.

8.4 Token-Kosten

Das Framework erfordert mehrere LLM-Calls pro Turn: mindestens einen Orchestrator-Call, einen Importance-Evaluierungs-Call pro Agent, einen Thinking-Call für den Gewinner-Agenten und einen Generierungs-Call für den vollständigen Turn.

Gegenüber der ursprünglichen Architektur ohne Thinking-Call entsteht damit ein zusätzlicher Call pro Turn – ausschließlich für den Gewinner, nicht für alle Agenten. Bei kostensensitiven Systemen ist zu prüfen, ob der Thinking-Call durch einen ausführlicheren System-Prompt substituiert werden kann; die konzeptuelle Trennung zwischen Vorüberlegung und Antwort geht dabei jedoch verloren (siehe Abschnitt 4.1.3). Option A (keine Backchannels) und sequenzielle statt parallele Evaluierung bleiben die wichtigsten Kostenhebel.

9 Zusammenfassung

Das Framework adressiert mit seinen fünf Ebenen fünf klar abgrenzbare Ursachen für unnatürliche Gesprächsverläufe in Multi-Agenten-Systemen. Ebene 1 verankert jede Persona durch priorisierte Überzeugungen und explizite Wissensgrenzen. Ebene 2 akkumuliert ein geteiltes Gedächtnis über den Gesprächsverlauf, getrennt nach Gesprächspartner, und ergänzt es um einen turn-lokalen Thinking-Block, der dem Gewinner-Agenten vor der Antwort eine explizite Verarbeitungsphase ermöglicht. Ebene 3 strukturiert den Sprecherwechsel durch ein Zwei-Mechanismen-System (CSSN und Self-Selection), das menschliches Turn-Taking nachbildet. Ebene 4 reguliert die Interaktionsdynamik durch Präferenzorganisation, Reparaturhierarchie und optionale Statusdifferenzierung. Ebene 5 koordiniert alle Ebenen über einen unsichtbaren Orchestrator, der Phasenwechsel erkennt, den Floor verwaltet und bei Gesprächspathologien eingreift.

Das Framework ist als iterativer Ausgangspunkt konzipiert – nicht als vollständige Lösung. **Ebene 3 (Turn-Taking) und Ebene 1 (Persona-Kern)** bieten den größten Hebel für sofortige Verbesserungen und sollten priorisiert implementiert und getestet werden. Ebene 2 ist unmittelbare Voraussetzung für konsistentes Turn-Taking und sollte parallel zu Ebene 3 eingeführt werden; der Thinking-Block (Schicht C) kann dabei zunächst weggelassen und in einer zweiten Iteration ergänzt werden. Die komplexeren Ebenen 4 und 5 können schrittweise folgen, wobei Block C (Statusdynamik) in frühen Iterationen weggelassen werden kann.

Tabelle 2 fasst Ebenen, Kernmechanismen und theoretische Primärquellen zusammen.

Literatur

Ebene	**Kernmechanismus**	**Primärquelle**
1 – Persona-Kern	Priorisierte Überzeugungen, Wissensgrenzen	Zhang et al. 2018; Goffman 1959
2 – Gedächtnis	Rohdaten-Buffer + paarweiser Kontextblock + Thinking-Block (pre-turn)	Park et al. 2023; Traum 2004
3 – Turn-Taking	CSSN (Agenten) + Orchestrator-First (alle Turns), 6 Regeln	Sacks et al. 1974; Nonomura & Mori 2025
4 – Interaktion	Präferenzorganisation, Reparaturhierarchie, Status	Pomerantz 1984; Schegloff et al. 1977
5 – Orchestrierung	Orchestrator-First, Phasenerkennung, Floor-Management, Eingreifregeln	Goffman 1981; Gu et al. 2022

Tabelle 2: Überblick: Ebenen, Mechanismen und theoretische Grundlagen

[1] Sacks, H., Schegloff, E. A. & Jefferson, G. (1974). A simplest systematics for the organization of turn-taking for conversation. *Language*, 50(4), 696–735. PDF: cs.columbia.edu

[2] Schegloff, E. A. (2007). *Sequence Organization in Interaction: A Primer in Conversation Analysis, Vol. 1.* Cambridge University Press. ISBN: 978-0-521-82572-6

[3] Schegloff, E. A. & Sacks, H. (1973). Opening up closings. *Semiotica*, 8(4), 289–327. PDF: web.stanford.edu

[4] Pomerantz, A. (1984). Agreeing and disagreeing with assessments. In Atkinson & Heritage (Eds.), *Structures of Social Action*, 57–101. Cambridge University Press. PDF: conversationanalysis.org

[5] Stivers, T. et al. (2009). Universals and cultural variation in turn-taking in conversation. *PNAS*, 106(26), 10587–10592. DOI: 10.1073/pnas.0903616106

[6] Schegloff, E. A., Jefferson, G. & Sacks, H. (1977). The preference for self-correction in the organization of repair in conversation. *Language*, 53(2), 361–382. DOI: 10.2307/413414

[7] Dingemanse, M. et al. (2015). Universal principles in the repair of communication problems. *PLoS ONE*, 10(9), e0136100. PDF: plos.org

[8] Maynard, D. W. (1980). Placement of topic changes in conversation. *Semiotica*, 30(3–4), 263–290. DOI: 10.1515/semi.1980.30.3-4.263

[9] Jefferson, G. (1984). On stepwise transition from talk about a trouble to inappropriately next-positioned matters. In Atkinson & Heritage (Eds.), *Structures of Social Action*, 191–222. Cambridge University Press.

[10] Goffman, E. (1959). *The Presentation of Self in Everyday Life*. Anchor Books/Doubleday. ISBN: 978-0-385-09402-3

[11] Goffman, E. (1981). *Forms of Talk*. University of Pennsylvania Press. ISBN: 978-0-8122-7790-6

[12] Clark, H. H. (1996). *Using Language*. Cambridge University Press. DOI: 10.1017/CBO9780511620539

[13] Edelsky, C. (1981). Who's got the floor? *Language in Society*, 10(3), 383–421. DOI: 10.1017/S004740450000885X

[14] Johnstone, Keith (1979). *Impro: Improvisation and the Theatre*. Faber & Faber. ISBN: 978-0-571-10989-0

[15] Zhang, S., Dinan, E., Urbanek, J., Szlam, A., Kiela, D. & Weston, J. (2018). Personalizing Dialogue Agents: I have a dog, do you have pets too? *Proceedings of ACL 2018*, 2204–2213. PDF: aclanthology.org/P18-1205.pdf

[16] Welleck, S., Weston, J., Szlam, A. & Cho, K. (2019). Dialogue Natural Language Inference. *Proceedings of ACL 2019*, 3731–3741. PDF: aclanthology.org/P19-1363.pdf

[17] Li, J., Galley, M., Brockett, C., Spithourakis, G. P., Gao, J. & Dolan, B. (2016). A Persona-Based Neural Conversation Model. *Proceedings of ACL 2016*, 994–1003. PDF: aclanthology.org/P16-1094.pdf

[18] Shao, Y., Li, L., Dai, J. & Qiu, X. (2023). Character-LLM: A Trainable Agent for Role-Playing. *EMNLP 2023*, 13153–13187. PDF: arxiv.org/pdf/2310.10158

[19] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P. & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST '23*. PDF: arxiv.org/pdf/2304.03442

[20] Traum, D. (2004). Issues in multiparty dialogues. In Dignum (Ed.), *Advances in Agent Communication*, LNCS 2922, 201–211. DOI: 10.1007/978-3-540-24608-4_12

[21] Nonomura, R. & Mori, H. (2025). Who speaks next? Multi-party AI discussion leveraging the systematics of turn-taking in Murder Mystery games. *Frontiers in Artificial Intelligence*, 8, 1582287. PDF: arxiv.org/pdf/2412.04937

[22] Li, J., Galley, M., Brockett, C., Gao, J. & Dolan, B. (2016). A Diversity-

Promoting Objective Function for Neural Conversation Models. *NAACL-HLT 2016*, 110–119. PDF: aclanthology.org/N16-1014.pdf

[23] Cho, H. & May, J. (2020). Grounding conversations with improvised dialogues. *Proceedings of ACL 2020*, 2398–2413. PDF: aclanthology.org/2020.acl-main.218.pdf

[24] Skantze, G. (2021). Turn-taking in conversational systems and human-robot interaction: a review. *Computer Speech & Language*, 67, 101178. DOI: 10.1016/j.csl.2020.101178

[25] Stokoe, E., Albert, S., Buschmeier, H. & Stommel, W. (2024). Conversation analysis and conversational technologies. *Discourse and Communication*, 18(6), 837–847. DOI: 10.1177/17504813241267118

[26] Gu, J.-C., Tao, C. & Ling, Z.-H. (2022). Who says what to whom: A survey of multi-party conversations. *IJCAI-22*, 5486–5493. PDF: ijcai.org/proceedings/2022/0768.pdf

[27] Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D. & Ghanem, B. (2023). CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. *NeurIPS 2023*. PDF: arxiv.org/pdf/2303.17760

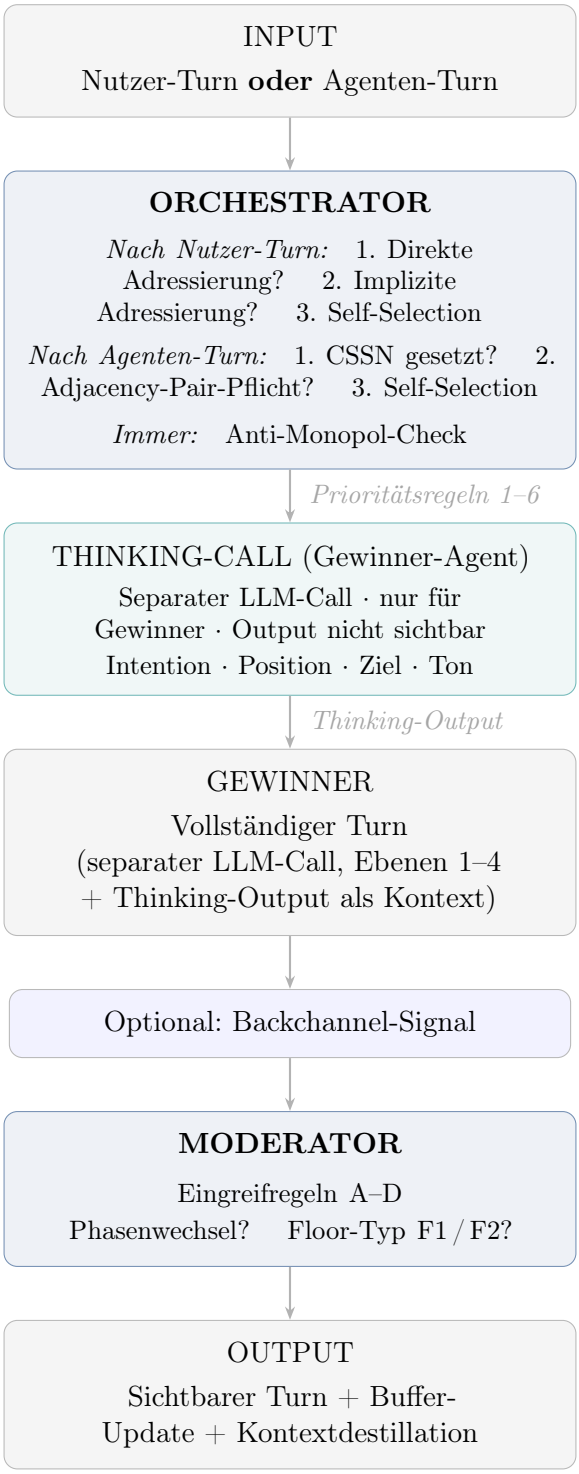

Abbildung 2: Entscheidungsfluss pro Konversations-Turn (Orchestrator-First-Architektur mit Thinking-Call). Der Orchestrator ist die invariante erste Verarbeitungsstufe; der Thinking-Call ist auf den Gewinner-Agenten beschränkt.

A Prompt-Anatomie

Dieser Anhang fasst zusammen, was konkret in welchem Prompt steht. Er dient als Referenz beim Aufbau der Prompt-Struktur und setzt voraus, dass die Ebenen 1–5 bekannt sind. Er beschreibt keine neuen Mechanismen, sondern ordnet die bereits definierten Bestandteile den jeweiligen Prompt-Instanzen zu.

Das System kennt genau zwei Prompt-Typen: den **System-Prompt des Diskussions-Agenten** (existiert N-fach, einmal pro Agent) und den **System-Prompt des Orchestrators** (existiert genau einmal). Hinzu kommen zwei schlanke Kurzform-Prompts: einer für die **Self-Selection-Evaluierung** und einer für den **Thinking-Call**, die jeweils nur für den Gewinner-Agenten bzw. alle Agenten pro Evaluierungsrunde instanziiert werden.

A.1 System-Prompt des Diskussions-Agenten

Jeder Diskussions-Agent erhält bei jedem LLM-Call denselben strukturellen Aufbau. Die Blöcke 1 und 3–5 sind statisch (ändern sich während des Gesprächs nicht); Block 2 ist dynamisch (wird bei jedem Turn neu befüllt).

```
=========================================================
BLOCK 1 -- PERSONA-KERN  [statisch, aus Ebene 1]
=========================================================
Du bist [Name], [Kurzbiografie in 2-3 Saetzen].

KERNUEBERZEUGUNGEN (in absteigender Prioritaet):
1. [Staerkste Ueberzeugung]
2. [Zweite Ueberzeugung]
3. [Dritte Ueberzeugung]

WISSENSGRENZEN:
- Du weisst nicht: [...]
- Du weigerst dich zu spekulieren ueber: [...]

SPRACHLICHER STIL:
- Satzlaenge: [kurz / mittel / lang]
- Typische Eroeffnung: [...]
- Verbotene Ausdruecke: [...]

STATUSDYNAMIK:
```

```
- Ausgangsstatus: [hoch / mittel / niedrig]

=========================================================
BLOCK 2 -- GEDAECHTNISBLOCK  [dynamisch, aus Ebene 2]
(wird bei jedem Turn neu berechnet und ersetzt)
=========================================================
GEDAECHTNIS von [Agent-Name]:

Was ich ueber den Nutzer weiss:
- [...]

Was ich ueber [Agent X] weiss:
- [...]
(wiederholt fuer jeden weiteren Agenten)

Offene Fragen:
- [...]

Letzter Gespraechsstand:
- Hauptthema: [1 Satz]
- Meine letzte Position: [1 Satz]

=========================================================
BLOCK 3 -- REAKTIONSREGELN  [statisch, aus Ebene 4]
=========================================================
REAKTIONS-TYPEN:
Bei Zustimmung:      Direkt, ohne Verzoegerung [...]
Bei Teilzustimmung:  Erst Uebereinstimmung, dann Differenz [...]
Bei Ablehnung:       Immer mit Verzoegerungssignal beginnen [...]

REPARATURMECHANISMEN:
Prioritaet 1 -- Selbstreparatur [...]
Prioritaet 2 -- Offene Klaerungsanfrage [...]
Prioritaet 3 -- Interpretierende Klaerung [...]

STATUSVERHALTEN:
[Abgeleitet aus Ausgangsstatus in Block 1]
Hoher Status:    Kurze, direkte Saetze. Keine Entschuldigungen. [...]
Mittlerer Status: [...]
Niedriger Status: [...]
```

```
=========================================================
BLOCK 4 -- AUSGABEANWEISUNG CSSN  [statisch, aus Ebene 3]
=========================================================
Haenge am Ende jeder Antwort folgenden Block an:

[METADATEN]
NAECHSTER_SPRECHER: [Name oder "Nutzer"]
ADJACENCY-PAIR-TYP: [...]
GRUND:              [1 Satz]

=========================================================
BLOCK 5 -- THEMENUEBERGANGSREGEL  [statisch, aus Ebene 3]
=========================================================
Themenwechsel nur ueber stepwise transition erlaubt.
Kein disjunktiver Schnitt ohne sprachliche Bruecke.
Mindestens 2 abgeschlossene Antwort-Paare pro Thema.
=========================================================
```

A.2 Kurzform-Prompt für die Self-Selection-Evaluierung

Dieser Prompt wird nicht dauerhaft vorgehalten, sondern nur in der Evaluierungsrunde instanziiert. Block 1 und Block 2 bleiben erhalten (damit der Agent seine Persona und seinen Kontext kennt), Blöcke 3–5 entfallen. Stattdessen tritt eine einzige Ausgabeanweisung hinzu:

```
[BLOCK 1 -- PERSONA-KERN, unveraendert]
[BLOCK 2 -- GEDAECHTNISBLOCK, aktueller Stand]

=========================================================
EVALUIERUNGSANWEISUNG
=========================================================
Generiere ausschliesslich den folgenden Block.
Keine inhaltliche Antwort. Kein Gespraechstext.

SELF-SELECTION-EVALUIERUNG:
IMPORTANCE:   [1-5]
ANTWORT-TYP:  [Frage / Zustimmung / Widerspruch /
               neue Information / Klaerung]
BEGRUENDUNG:  [1 Satz]
```

```
SPRECHE ICH:  [Ja / Nein / Warte]
=========================================================
```

A.3 Kurzform-Prompt für den Thinking-Call

Dieser Prompt wird ausschließlich für den vom Orchestrator gewählten Gewinner-Agenten instanziiert – einmal pro Turn, nach der Orchestrator-Entscheidung und vor dem eigentlichen Turn-Call. Block 1 und Block 2 bleiben erhalten; Blöcke 3–5 entfallen. Der Output wird nicht in den Rohdaten-Buffer geschrieben und ist für andere Agenten nicht sichtbar. Er fließt als vorgeschalteter Assistant-Turn in die Message-History des nachfolgenden Turn-Calls ein.

```
[BLOCK 1 -- PERSONA-KERN, unveraendert]
[BLOCK 2 -- GEDAECHTNISBLOCK, aktueller Stand]

=========================================================
THINKING-ANWEISUNG
=========================================================
Bevor du antwortest, arbeite folgendes durch.
Kein Gespraechstext. Nur dieser interne Block.

THINKING:
WAS_WURDE_GESAGT:   [1 Satz -- Inhalt des letzten Turns]
INTENTION_DAHINTER: [1 Satz -- was will der Sprecher damit erreichen?]
MEINE_POSITION:     [Zustimmung / Teilzustimmung / Ablehnung -- und
    warum,
                     gemaess meinen Kernueberzeugungen]
MEIN_ZIEL:          [1 Satz -- was will ich mit meiner Antwort erreichen
    ?]
TON:                [gemaess Status und aktueller Gespraechsphase]
=========================================================
```

A.4 System-Prompt des Orchestrators

Der Orchestrator-Prompt ist vollständig statisch – mit einer Ausnahme: Block 2 (Gesprächsstand) wird bei jedem Call aktualisiert.

```
=========================================================
BLOCK 1 -- ROLLENDEFINITION  [statisch]
```

```
=========================================================
Du bist der unsichtbare Koordinator dieses Gespraechs.
Du erscheinst nie als Gespraechsteilnehmer.
Du hast keine Meinung zum Gespraechsinhalt.
Deine einzige Aufgabe ist die Steuerung des Gespraechsablaufs.

Die Diskussions-Agenten in diesem Gespraech sind:
- [Agent-Name 1]
- [Agent-Name 2]
- [... fuer jeden weiteren Agenten]

=========================================================
BLOCK 2 -- GESPRAECHSSTAND  [dynamisch, bei jedem Call aktualisiert]
=========================================================
Aktuelle Gespraechsphase: [Opening / Development /
                           Closing-Implicative / Closing]
Aktueller Floor-Typ:      [F1 / F2]

Turn-Verlauf (komprimiert):
- Turn N-4: [Sprecher]: [1-Satz-Zusammenfassung]
- Turn N-3: [Sprecher]: [...]
- Turn N-2: [Sprecher]: [...]
- Turn N-1: [Sprecher]: [...]
- Turn N:   [Sprecher]: [...]

Letzter CSSN-Block (falls vorhanden):
  NAECHSTER_SPRECHER: [...]
  ADJACENCY-PAIR-TYP: [...]

Importance-Scores dieser Runde (falls Self-Selection):
- [Agent A]: [Score] / [Antwort-Typ]
- [Agent B]: [Score] / [Antwort-Typ]

Monopol-Zaehler:
- [Agent A]: [Anzahl aufeinanderfolgender Turns]
- [Agent B]: [...]

=========================================================
BLOCK 3 -- AUSWAHLLOGIK  [statisch, aus Ebene 3 und 5]
=========================================================
```

```
Wende die folgenden Regeln in dieser Reihenfolge an:

Nach Nutzer-Turn:
  1. Direkte Adressierung?   → Dieser Agent hat Vorrang.
  2. Implizite Adressierung? → Dieser Agent hat Vorrang.
  3. Self-Selection: Regeln 3-6 anwenden.

Nach Agenten-Turn:
  1. CSSN gesetzt? → Adjacency-Pair-Pflicht pruefen.
  2. Kein CSSN?    → Self-Selection: Regeln 3-6 anwenden.
  Immer: Anti-Monopol-Pruefung (Score halbieren bei 2+ Turns).

Regeln 3-6 (Self-Selection):
  3. Anti-Monopol: Score halbieren bei Dominanz.
  4. Diversity:    Abweichenden Antwort-Typ bevorzugen.
  5. Score:        Hoechsten Importance-Score waehlen.
  6. Tiebreaker:   Gleichstand → zufaellig.

Phasenwechsel-Bedingungen:
  Opening → Development:
    Alle Agenten mindestens 1x zu Wort
  Development → Closing-Implicative:
    Beitraege kurz + Wiederholungen + keine neuen Informationen
  Closing-Implicative → Closing:
    Moderator leitet ein

Floor-Wechsel-Bedingungen:
  F1 → F2: Beitraege sehr kurz + hohe Zustimmungsrate
  F2 → F1: Kontroverse entsteht

=========================================================
BLOCK 4 -- EINGREIFREGELN  [statisch, aus Ebene 5]
=========================================================
[A] Ein Agent > 2 Turns in Folge ohne Reaktion.
    "[MODERATION]: [Agent B], was denkst du ueber [Xs] Punkt?"

[B] Thema > 5 Turns ohne Fortschritt.
    "[MODERATION]: Wir scheinen uns im Kreis zu drehen.
     [Agent X], kannst du einen neuen Aspekt einbringen?"
```

```
[C] Nutzer-Frage wurde ignoriert.
    "[MODERATION]: [Agent Y], [Nutzer] hat gefragt, ob..."

[D] Direktkonflikt ohne Aufloesung nach 3 Turns.
    "[MODERATION]: Lasst uns einen Schritt zuruecktreten.
     Wo seid ihr euch eigentlich einig?"

=======================================================
BLOCK 5 -- AUSGABEFORMAT  [statisch]
=======================================================
Gib nach jeder Auswertung genau folgendes aus:

NAECHSTER_SPRECHER: [Name]
GRUND:              [1 Satz]
GESPRAECHSPHASE:    [aktuell / gewechselt zu: ...]
FLOOR-TYP:          [F1 / F2]
EINGRIFF:           [Nein / Ja: Regel [A/B/C/D], Text: "..."]
=======================================================
```

A.5 Übersicht: Welcher Block gehört wohin?

Tabelle 3 fasst die Zuordnung aller Framework-Bestandteile zu den Prompt-Typen zusammen.

Ebene	Bestandteil	Diskussions-Agent	Orchestrator
1	Persona-Kern (Biografie, Überzeugungen, Stil)	Block 1, statisch	–
1	Ausgangsstatus (für Ebene 4)	Block 1, statisch	–
2	Komprimierter Kontextblock (pro Agent)	Block 2, dynamisch	–
2	Thinking-Block (Gewinner-Agent, pre-turn)	Kurzform-Prompt	–
2	Gesprächsstand (agenten-übergreifend)	–	Block 2, dynamisch
3	CSSN-Ausgabeanweisung	Block 4, statisch	–
3	Self-Selection-Evaluierung	Kurzform-Prompt	–
3	Auswahlregeln 1–6	–	Block 3, statisch
3	Themenübergangsregel	Block 5, statisch	–
4	Präferenzorganisation (Block A)	Block 3, statisch	–
4	Reparaturmechanismen (Block B)	Block 3, statisch	–
4	Statusverhalten (Block C)	Block 3, statisch	–
5	Rollendefinition Orchestrator	–	Block 1, statisch
5	Phasendefinitionen & Wechselbedingungen	–	Block 3, statisch
5	Floor-Management-Regeln	–	Block 3, statisch
5	Eingreifregeln A–D	–	Block 4, statisch
5	Ausgabeformat Orchestrator	–	Block 5, statisch

Tabelle 3: Zuordnung aller Framework-Bestandteile zu den Prompt-Typen. Ein Strich (–) bedeutet: dieser Bestandteil ist im jeweiligen Prompt nicht vorhanden.